

RUGID for Dummies

Antigravity

Contents

RUGID for Dummies	2
The Definitive Guide to the Relativistic Unified Graphic Interface	
Definition Engine	2
Table of Contents	3
Part I: The Philosophy of RUGID	3
Part II: Speaking RUGID (RDF)	3
Part III: The Ontology of Space	3
Part IV: Advanced Mechanics	3
Preface	4
Chapter 1: Foundations	4
Why RUGID?	4
The Relativistic Mindset	4
Architecture	4
The Runtime Loop	4
Temporal Memory & The Beacon	5
Spatial Hashing	5
Chapter 2: RDF for Dummies	5
1. The Anatomy of a Line	5
Example	5
2. Hierarchy and Indentation	5
3. Element Types	6
window	6
pane	6
widget	6
text	6
shape3d	6
4. Attributes in Detail	6
Geometry (origin , extent)	6
Appearance (fill , color)	7
Text (align , baseline)	7
5. Animation: The Tilde (~)	7
6. Comments	7

7. A Complete Example	7
Troubleshooting	7
Chapter 3: The Ontology of Space	8
1. Primary vs. Secondary	8
2. Relative Sizing	8
Percent(f32)	8
Flex(f32)	8
Aspect(f32)	8
3. LocalSpace	8
4. Position Modes	9
5. Text Sizing	9
Chapter 4: 3D in a 2D World	9
1. The Camera and the Canvas	9
2. Shape3D	9
Supported Shapes	9
Attributes	10
3. Rotations and Orbits	10
Declarative Rotation	10
Orbit States	10
4. Physics	10
Properties	10
The Physics Loop	10
5. Integrating 3D and Physics	11
Chapter 5: Advanced Mechanics	11
1. Inter-Module Ballistic Missiles (IMBM)	11
The Signal	11
The Trajectory	11
2. The Beacon	11
3. Temporal Memory	12
Time Travel	12
The Delta Frame	12
4. Writing Custom Logic	12
Conclusion	12

RUGID for Dummies

The Definitive Guide to the Relativistic Unified Graphic Interface Definition Engine

Version: 1.0

Author: Antigravity

Target Audience: Developers, Designers, and anyone who wants to escape the tyranny of absolute coordinates.

Table of Contents

Part I: The Philosophy of RUGID

1. **Introduction: Why RUGID?**
 - The Problem with Absolute Coordinates
 - The Relativistic Mindset
 - Cells, Not Pixels
2. **The Architecture of Reality**
 - The Runtime Loop
 - Temporal Memory & The Beacon
 - Spatial Hashing

Part II: Speaking RUGID (RDF)

3. **RDF for Dummies**
 - What is RDF? (Not that RDF)
 - The Syntax: Colons, Brackets, and Tabs
 - The Hierarchy: Windows, Panes, and Widgets
 - Attributes: Styling your Reality
 - Animation: The Tilde Operator (~)

Part III: The Ontology of Space

4. **Understanding LocalSpace**
 - Primary vs. Secondary Axes
 - Relative Sizing (% , `flex`, `aspect`)
 - Stacking and Layout
5. **3D in a 2D World**
 - The Camera and the Canvas
 - Shapes: From Cubes to Mercedes W14s
 - Rotations and Orbits

Part IV: Advanced Mechanics

6. **Physics and Motion**
 - Mass, Velocity, and Collisions
 - The Physics System
 7. **Inter-Module Ballistic Missiles (IMBM)**
 - Signal Propagation
 - Event Handling
-

Preface

Welcome to RUGID. If you are reading this, you are likely tired of CSS **z-index** wars, confused by **flex-grow** vs **flex-shrink**, or just want to build user interfaces that feel like living organisms rather than static documents.

RUGID is not just a UI engine; it is a simulation. It treats every UI element as a “Cell” living in a relativistic space. There is no “top-left” of the screen. There is only “relative to my parent”.

This book will guide you from “Hello World” to building complex, 3D-accelerated, physics-enabled interfaces using the RUGID Definition Format (RDF).

Buckle up. It’s going to be a relativistic ride.

Chapter 1: Foundations

Why RUGID?

In the beginning, there were pixels. And they were absolute. And it was bad. Then came vectors. And they were scalable. But they were still positioned absolutely. Then came constraints and flexboxes. And they were better, but complex.

RUGID (Relativistic Unified Graphic Interface Definition) takes a different approach. It posits that **nothing exists in isolation**. Every element exists only in relation to its container.

The Relativistic Mindset

In RUGID, you never ask “Where is this button?”. You ask “Where is this button *relative to its parent*?”. - A button is not at **x=100**. - A button is at **origin_s=0.1** (10% of the way across the secondary axis).

This means your UI is **inherently responsive**. You don’t write media queries to resize elements. The elements resize themselves because they are defined as percentages of their parents.

Architecture

RUGID is not just a layout engine. It is a **Simulation Runtime**.

The Runtime Loop

Most UI frameworks are event-driven. They wait for a click, then update. RUGID runs a continuous loop (the **tick**). 1. **Poll Events**: Input, Resize. 2. **Update Physics**: Apply forces, velocity. 3. **Update Logic**: Process signals (IMBM). 4. **Advance Time**: Update temporal memory. 5. **Render**: Draw the state.

This game-loop architecture allows for smooth, physics-based animations and interactions that feel “alive”.

Temporal Memory & The Beacon

RUGID remembers the past. The **Temporal Memory** stores the state of every cell over time. The **Beacon** watches for signals. When a signal is fired (e.g., a button press), the Beacon can trigger a “Time Dilated” response, or simply update the state.

Spatial Hashing

To handle thousands of cells efficiently, RUGID uses **Spatial Hashing**. It divides the screen into buckets and only processes interactions for cells in the relevant buckets. This makes hit-testing and collision detection extremely fast.

Chapter 2: RDF for Dummies

RDF stands for **RUGID Definition Format**. It is NOT the Resource Description Framework used in the semantic web. It is a concise, indentation-based, relativistic scene description language designed specifically for RUGID.

If you know HTML or YAML, you’re halfway there. But RDF is stricter, cleaner, and built for speed.

1. The Anatomy of a Line

Every line in an RDF file describes a single **Node** (or Cell) in the ontology.

`element::attribute1[value]::attribute2[value]`

- **Element**: The type of object (e.g., `window`, `pane`, `widget`).
- `:::`: The separator between the element type and its attributes.
- **Attribute**: A key-value pair.
- `::`: The separator between attributes.
- `[ ]`: Encloses the attribute value.

Example

```
widget:::id[my_button]::origin[0.5,0.5]::extent[0.2,0.1]::fill[#FF0000]
```

This defines a `widget` with ID `my_button`, centered in its parent (0.5,0.5), taking up 20% width and 10% height, filled with red.

2. Hierarchy and Indentation

RUGID is hierarchical. Children are defined by **indentation**. You can use **Tabs** or **4 Spaces**. Be consistent.

```

window:::title[My App]
  pane:::id[main_layout]
    widget:::id[header]
    widget:::id[content]
      text:::content[Hello World]

```

In this example: - `window` is the root. - `pane` is a child of `window`. - `header` and `content` are children of `pane`. - `text` is a child of `content`.

3. Element Types

`window`

The root of your application. - **Attributes:** `title`, `sdims` (screen dimensions, mostly for initial setup). - **Example:** `window:::title[RUGID Demo]`

`pane`

A container for layout. Panes usually don't have visual content themselves (unless you add a `fill`), but they organize children. - **Attributes:** `id`, `parent`, `origin`, `extent`, `fill`. - **Example:** `pane:::id[sidebar]::extent[0.2,1.0]` (20% width, 100% height).

`widget`

A visual element. Can be a button, a card, or a background. - **Attributes:** `id`, `parent`, `origin`, `extent`, `fill`. - **Example:** `widget:::fill[#00FF00]`

`text`

Renders text. - **Attributes:** `content`, `size` (relative to parent height!), `color`, `align`, `baseline`, `font`, `weight`. - **Example:** `text:::content[Click Me]::size[0.5]::color[white]` - `size[0.5]` means the font height is 50% of the parent cell's height. This is crucial for responsive design.

`shape3d`

A 3D object. - **Attributes:** `type` (cube, sphere, mercedes), `pos` (x,y,z), `rot` (x,y,z), `scale`. - **Example:** `shape3d:::type[sphere]::pos[0,0,-5]`

4. Attributes in Detail

Geometry (`origin`, `extent`)

RUGID uses **Relative Coordinates**. - `origin[x,y]`: Position relative to parent top-left. 0.0 is 0%, 1.0 is 100%. - `extent[w,h]`: Size relative to parent size. - `extent[0.5, 0.5]`: 50% width, 50% height. - `extent[flex, 1.0]`: Fills remaining width, 100% height.

Appearance (fill, color)

- **fill:** Background color. Supports Hex (`#RRGGBB`) and RGB (`rgb(r,g,b)`).
- **color:** Text color.

Text (align, baseline)

- **align:** start, center, end.
- **baseline:** top, middle, bottom.

5. Animation: The Tilde (~)

RUGID has built-in support for declarative animation using the `~` operator.

```
shape3d:::type[cube]::rotatey~90
```

This tells the engine: “Animate `rotatey` to 90 degrees”. The engine handles the interpolation automatically.

You can also specify duration (in some contexts, though currently defaults are often used).

6. Comments

Lines starting with `#` are comments.

```
# This is a comment
```

```
window:::title[App] # Inline comments are not fully supported yet, put them on new lines
```

7. A Complete Example

Here is `demo.rdf`:

```
window:::title[RDF Demo]::sdims[100,100,100]
  # Main layout pane
  pane:::id[mainpane]::parent[RDF Demo]::origin[0,0]::extent[1.0,1.0]::fill[#FF0000]
    # A centered text widget
    widget:::id[center_box]::origin[0.4,0.4]::extent[0.2,0.2]::fill[white]
      text:::content[Hello]::size[0.8]::align[center]::baseline[middle]::color[black]
```

Troubleshooting

- **Empty Screen?** Check if your elements have `fill` or `content`. A transparent pane is invisible.
- **Wrong Layout?** Check indentation. One wrong space can detach a child from its parent.
- **Parsing Errors?** Ensure you use `[` and `]` for values. `origin[0,0]` is correct. `origin=0,0` is wrong.

Chapter 3: The Ontology of Space

RUGID's coordinate system is unique. It abstracts “Width” and “Height” into **Primary** and **Secondary** axes, depending on the **Stack Direction**.

1. Primary vs. Secondary

- **StackDirection::Primary** (Vertical):
 - **Primary Axis:** Y (Height)
 - **Secondary Axis:** X (Width)
 - Children stack vertically.
- **StackDirection::Secondary** (Horizontal):
 - **Primary Axis:** X (Width)
 - **Secondary Axis:** Y (Height)
 - Children stack horizontally.

This abstraction allows you to change a layout from Row to Column simply by changing the **StackDirection**, without rewriting all your dimensions.

2. Relative Sizing

Sizes in RUGID are **RelativeSize** enums:

Percent(f32)

The standard. 0.5 means 50% of the parent's dimension on that axis. - If parent width is 1000px, **Percent(0.5)** is 500px.

Flex(f32)

Used in Stacks. Fills remaining space. - If you have 3 items with **Flex(1.0)**, they share the space equally (33% each). - If one has **Flex(2.0)** and others **Flex(1.0)**, the first gets 50%, others 25%.

Aspect(f32)

Maintains aspect ratio relative to the *other* axis. - **Aspect(1.0)**: Square. - **Aspect(1.77)**: 16:9 Widescreen. - Useful for keeping images or shapes correct regardless of parent resizing.

3. LocalSpace

Every node has a **LocalSpace**. This defines its geometry *before* it is resolved to absolute coordinates.

```
pub struct LocalSpace {  
    pub origin_p: f32,      // Origin on Primary axis  
    pub origin_s: f32,      // Origin on Secondary axis
```

```

    pub extent_p: RelativeSize, // Size on Primary axis
    pub extent_s: RelativeSize, // Size on Secondary axis
    pub rotation_deg: f32, // Rotation
    pub clips_children: bool, // Overflow hidden?
    // ...
}

```

4. Position Modes

- **Stacked:** The default. The element is positioned automatically by the parent’s stack logic. `origin` is ignored.
- **Absolute:** The element is positioned explicitly using `origin`. It floats above the stack.

5. Text Sizing

Text size is special. It is defined relative to the **Height** of its container. `font_size: 0.5` means “Half the height of the cell”. This ensures text scales perfectly with the UI. No more “12px” text on a 4K screen.

Chapter 4: 3D in a 2D World

RUGID is primarily a 2D UI engine, but it lives in a 3D world. Every cell technically has a Z-coordinate, and specific cells can be fully 3D objects.

1. The Camera and the Canvas

The RUGID “Canvas” is a 2D plane at `z=0`. 3D objects are projected onto this plane. Currently, the projection is **Orthographic** (mostly), meaning objects don’t get smaller as they get further away, but this may change to Perspective in future versions.

2. Shape3D

The `shape3d` element allows you to embed 3D geometry into your UI.

```
shape3d::type[cube]::pos[0,0,0]::rot[45,45,0]::scale[1.0]
```

Supported Shapes

- **Primitives:** `cube`, `sphere`, `pyramid`, `tetrahedron`.
- **Complex:** `mercedes` (Mercedes W14 F1 car), `f1` (Generic F1), `jabulani` (Soccer ball).
- **Custom:** You can load `.obj` files (experimental).

Attributes

- `pos[x,y,z]`: Position relative to the cell center.
- `rot[x,y,z]`: Rotation in degrees (Euler angles).
- `scale[s]`: Uniform scaling.
- `opacity[alpha]`: Transparency (0.0 to 1.0).

3. Rotations and Orbits

Static 3D objects are boring. RUGID provides two ways to make them move:

Declarative Rotation

Set a rotation speed in degrees per frame (or second).

```
shape3d:::type[cube]::rotatey~1.0
```

This spins the cube around the Y-axis.

Orbit States

For more complex movement, cells can have an `Orbit3DState`. This defines a satellite-like behavior where the object orbits a center point.

```
pub struct Orbit3DState {  
    pub radius: f32,  
    pub theta: f32, // Horizontal angle  
    pub phi: f32,   // Vertical angle  
    pub speed: f32,  
}
```

4. Physics

RUGID includes a rigid-body physics engine. Any cell can be given physical properties.

```
widget:::id[bouncy_box]::physics[mass:1.0, restitution:0.8]
```

Properties

- **Mass**: How heavy the object is. 0.0 means infinite mass (static).
- **Velocity**: Initial speed [`vx`, `vy`].
- **Restitution**: Bounciness (0.0 = brick, 1.0 = superball).
- **Friction**: How much it slows down when sliding.

The Physics Loop

Every tick, the engine: 1. Applies Gravity (if enabled). 2. Updates Velocity based on forces. 3. Updates Position based on velocity. 4. Detects Collisions between cells. 5. Resolves Collisions (bouncing).

This allows you to create UIs where windows fall, bounce, and collide with each other.

5. Integrating 3D and Physics

You can combine them. A **shape3d** can have physics. Imagine a 3D dice rolling across your UI. - The **Visual** is a 3D Cube. - The **Physical** body is a bounding box. - The **Simulation** handles the trajectory.

This is the power of RUGID: High-fidelity visuals driven by a robust simulation.

Chapter 5: Advanced Mechanics

Now we enter the realm of the esoteric. RUGID isn't just about drawing boxes; it's about managing information flow through time and space.

1. Inter-Module Ballistic Missiles (IMBM)

Most systems have “Events”. RUGID has **IMBMs**. Why “Ballistic Missiles”? Because signals in RUGID have a **Trajectory** and a **Payload**, and they can be “launched” across the application.

The Signal

A signal is a packet of information.

```
pub struct Signal {  
    pub id: SignalId,  
    pub source: CellId,  
    pub payload: Payload,  
    pub strength: f32,  
}
```

The Trajectory

Signals don't just magically appear everywhere. They propagate. - **Direct**: Target a specific Cell ID. - **Broadcast**: Send to all children. - **Bubble**: Send to parent. - **Spatial**: Send to all cells within radius R.

This allows for localized events. A button click might only “shake” the neighboring buttons, rather than alerting the global state.

2. The Beacon

The **Beacon** is the central nervous system. It tracks the version of every signal. When you subscribe to a signal, you are telling the Beacon: “Wake me up when this changes”.

```
// In your update loop
if beacon.track(my_signal_id, last_seen_version) {
    // Signal has changed!
    let new_data = beacon.read(my_signal_id);
    // React...
}
```

3. Temporal Memory

RUGID remembers what happened. **Temporal Memory** is a ring buffer of the application state.

Time Travel

Because we store history, we can implement “Undo” or “Replay” trivially. We can also do **Time Dilation**. If a heavy calculation is needed, we can “slow down” time for that specific cell, letting it update less frequently, while the rest of the UI runs at 60 FPS.

The Delta Frame

We don’t store snapshots. We store **Deltas**. **DeltaFrame** contains only what changed. - **delta_pos**: Movement. - **delta_size**: Growth/Shrinkage. - **delta_color**: Color shift.

This makes the memory footprint surprisingly small.

4. Writing Custom Logic

To make your app interactive, you implement the **Projector** trait.

```
impl Projector<InputState> for MyButton {
    fn project(&self, input: InputState) -> Projection {
        if input.is_clicked(self.bounds) {
            // Launch an IMBM!
            return Projection::Signal(Signal::new("BUTTON_CLICKED"));
        }
        Projection::None
    }
}
```

The **Projector** takes the current state and “projects” a possible future (a **Signal**). The runtime then decides if that future becomes reality.

Conclusion

You have reached the end of **RUGID for Dummies**. You now understand: - Why relative coordinates rule. - How to write RDF. - How to place 3D objects.

- How the physics and signal systems work.
- Go forth and build interfaces that defy the absolute.